

TECHNICAL REPORT

IoT / EMBEDDED SYSTEMS

Project 4: ESP-NOW Two-Way Wireless Interlock System

Daily Technical Report — Day 10

Field	Detail
Platform	ESP32 & ESP32-S3 (Dual Board Pair) · Arduino Framework · PlatformIO
Subsystem	ESP-NOW Wireless Peer-to-Peer Communication Protocol
Prepared By	Malak Alsharqawi
Partner	Dana natsheh
Submission	GitHub Repository Link + Wiring Schematic
Prepared For	IoT / Embedded Systems Training Program

Confidential — Internal Training Documentation

Contents

1. Daily Overview	3
1.1 Summary	3
1.2 Tasks Completed Today	3
2. Project 4: ESP-NOW Two-Way Wireless Interlock System	4
2.1 Objective	4
2.2 Project Description	4
2.3 Technical Requirements	4
1. Peer Setup	4
2. Data Structure	4
3. Callbacks	4
4. Fail-Safe Timeout	4
2.4 Repository File Structure	4
2.5 Definition of Done	5
3. What is ESP-NOW?	6
3.1 Overview & Motivation	6
Why ESP-NOW fits this project	6
3.2 ESP-NOW vs. Traditional Wi-Fi	6
3.3 MAC Addresses and Peers	6
3.4 How ESP-NOW Works — Communication Flow	7
4. System Architecture	8
4.1 Peer-to-Peer (Transceiver) Design	8
4.2 The Message Data Structure	8
4.3 Sending Data — <code>esp_now_send()</code>	8
4.4 Receiving Data & Callback Functions	8
OnDataSent() — delivery status (optional, useful for debugging)	9
OnDataRecv() — essential, this is where the interlock logic reacts	9
5. Fail-Safe Design	10
5.1 Timeout Logic (2-Second Rule)	10
5.2 Why <code>millis()</code> Instead of <code>delay()</code>	10
5.3 Heartbeat Messages	10
7. Wiring & Hardware Setup	11
8. Key Technical Concepts / Lessons Learned	12
ESP-NOW	12
Callbacks	12
Data Structures	12
Fail-Safe	12
9. Schematic & Photos	13
Schematic	13
Photos	13
10. References	15

1. Daily Overview

1.1 Summary

Day 10 shifted the training program from single-board motor control into true multi-board wireless collaboration. Working in pairs, each intern's ESP32 was configured to communicate directly with a partner board using ESP-NOW, a connectionless, router-free wireless protocol built into the ESP32's Wi-Fi radio. The deliverable is a two-way safety interlock: each board reads its own local IR sensor and uses that reading to command the partner board's motor, while simultaneously accepting commands sent back the other way. Neither board acts as a server, client, access point, or router — both boards are simultaneous senders and receivers (transceivers) in a symmetric peer-to-peer link.

The session covered the theory of ESP-NOW (what it is, why it differs from conventional Wi-Fi, and how MAC-address-based peers replace IP addressing), followed by the practical build: defining a shared packet structure, registering send/receive callbacks, wiring the interlock logic between the IR sensor and the partner's motor, and — critically — implementing a 2-second communication-loss fail-safe so that a dropped connection always resolves to the safest possible state (motor stopped).

1.2 Tasks Completed Today

- Studied ESP-NOW fundamentals: connectionless protocol, MAC-address addressing, no router/AP/internet dependency
- Compared ESP-NOW against traditional Wi-Fi/TCP-IP communication (latency, power, packet size, addressing)
- Reviewed the required ESP-NOW initialization order: Wi-Fi Station Mode → `esp_now_init()` → Add Peer → Register Callbacks
- Defined the shared `message_t` / `Message` struct (`senderID`, `runMotor`, `timestamp`) used by both boards
- Implemented `esp_now_send()` calls and non-blocking `OnDataSent()` / `OnDataRecv()` callback functions
- Recorded both boards' MAC addresses (ESP32: 78:42:1C:6C:13:68, ESP32-S3: 9C:13:9E:A8:6C:B4) and used them to register peers
- Designed the 2-second fail-safe timeout using `millis()` so a lost connection stops the motor automatically
- Discussed heartbeat-style periodic transmission so the receiver can distinguish "alive" from "disconnected"
- Refactored the communication layer into a reusable `ESPNow` C++ class (`begin`, `addPeer`, `send`, `onReceive`, `onSend`)
- Restructured the `PlatformIO` project into modular header/source pairs: `config.h`, `ESPNow.h/.cpp`, `L298N.h/.cpp`, `Message.h`, `main.cpp`
- Validated two-way packet exchange between the ESP32 and ESP32-S3 boards using a `recv/send bench test`

2. Project 4: ESP-NOW Two-Way Wireless Interlock System

2.1 Objective

Work in pairs to build a two-way wireless interlock system using ESP-NOW, where each intern's IR sensor controls the other intern's motor — with no Wi-Fi router, access point, or internet connection involved at any point in the data path.

2.2 Project Description

Each pair of interns sets up an ESP-NOW peer-to-peer link between two ESP32 boards. Neither board is a central hub; both act as simultaneous senders and receivers:

- Intern A reads their local IR sensor. If NO obstacle is detected, Intern A transmits a RUN command over ESP-NOW to Intern B's motor. If an obstacle IS detected, Intern A transmits a STOP command.
- Intern B performs the identical logic in reverse — their local IR sensor controls Intern A's motor.

```
[Intern A ESP32] --- (IR Sensor) ===[ ESP-NOW Wireless ]==> (Motor) [Intern B ESP32]
[Intern A ESP32] <--- (Motor) <===[ ESP-NOW Wireless ]== (IR Sensor) [Intern B ESP32]
```

This makes the system a genuine safety interlock: each board's sensor input becomes the other board's actuator command, and the wireless link must be monitored continuously for both directions.

2.3 Technical Requirements

1. Peer Setup

MAC addresses must be hardcoded (or dynamically paired) into the `esp_now_peer_info_t` structure so each board knows exactly which partner it is allowed to exchange packets with.

2. Data Structure

A clean, shared struct defines every state message sent over the link:

```
typedef struct
{
    uint8_t senderID; // which board sent this message
    bool runMotor; // true = run, false = stop
    uint32_t timestamp; // millis() at time of send
} Message;
```

3. Callbacks

Non-blocking `OnDataSent()` and `OnDataRecv()` callback functions handle transmission confirmation and incoming packets respectively, without ever calling `delay()` or performing heavy work inside the callback itself.

4. Fail-Safe Timeout

Safety Rule

If an ESP32 does not receive an ESP-NOW packet from its partner for more than 2 seconds (connection lost, or partner powered off), it must automatically stop its local motor.

2.4 Repository File Structure

The PlatformIO project was reorganized to separate wireless communication, motor control, and message definitions into independent, testable modules:

```
TASK1_DAY10/
├── include/
│   └── config.h          <- Pins, peer MACs, PWM & timeout constants
```

```

├── ESPNow.h      <- ESP-NOW class declaration
├── L298N.h      <- Motor driver class declaration
├── Message.h    <- Shared packet struct (senderID, runMotor, timestamp)
├── lib/         <- Third-party / custom libraries
├── src/
│   ├── ESPNow.cpp    <- ESP-NOW class implementation (init, peer, send, callbacks)
│   ├── L298N.cpp    <- Motor driver implementation
│   └── main.cpp      <- setup()/loop(): read IR, send state, apply received state
├── test/
│   ├── README
│   └── recv_send_test_one_... <- Bench test for two-way packet exchange
└── .gitignore

```

2.5 Definition of Done

#	Checklist Item
1	Both ESP32s pair successfully via ESP-NOW without needing an external Wi-Fi router.
2	Blocking Intern A's IR sensor immediately stops Intern B's motor, and vice versa.
3	Disconnecting power from one ESP32 causes the other ESP32 to trigger the 2-second fail-safe timeout and safely shut down its motor.
4	A clear dual-MCU wiring diagram is included in the repository.
5	Submission includes the GitHub repository link and wiring schematic.

3. What is ESP-NOW?

3.1 Overview & Motivation

ESP-NOW is a connectionless wireless communication protocol developed by Espressif for the ESP32 and ESP8266 family of chips. It piggybacks on the same 2.4 GHz radio hardware used for Wi-Fi, but it bypasses the entire Wi-Fi/TCP-IP stack. There is no router, no access point, no DHCP negotiation, no IP address, and no internet connection anywhere in the path. Instead, two ESP32 boards exchange small packets of data directly, addressed purely by MAC address.

The simplest mental model is a walkie-talkie: both devices already know each other's identity (their MAC address), one device keys up and sends a message, and the other immediately receives it — with no dispatcher, tower, or network operator in between.

Why ESP-NOW fits this project

- Very low latency — critical for a real-time safety interlock where a blocked IR sensor must stop a motor almost instantly
- No Wi-Fi router required — the two boards can be carried anywhere and still talk to each other
- Low power consumption — no need to maintain an association with an access point
- Native support for one-to-one (exactly this project) and one-to-many topologies
- Small, fixed-size packets are a natural fit for a compact state struct like Message

3.2 ESP-NOW vs. Traditional Wi-Fi

Feature	ESP-NOW	Traditional Wi-Fi
Router required	No	Yes
Addressing	MAC address	IP address
Latency	Very low	Higher
Protocol stack	Lightweight, direct	Full TCP/IP
Packet size	Small, fixed	Large packets supported
Power consumption	Low	Higher

Instead of the familiar path:

```
ESP32  --->  Router  --->  ESP32
```

ESP-NOW communicates directly:

```
ESP32  <----->  ESP32
```

3.3 MAC Addresses and Peers

Every ESP32 has a unique hardware MAC address (e.g., 24:6F:28:AB:CD:EF), retrievable in code via `WiFi.macAddress()`. ESP-NOW sends packets directly to this address — think of it as a house address, versus an IP address which is more like a mailbox that can change. For this project the two partner boards' MAC addresses were recorded ahead of time:

Board	MAC Address
ESP32	78:42:1C:6C:13:68
ESP32-S3	9C:13:9E:A8:6C:B4

A peer is another ESP32 that a device is explicitly allowed to talk to — similar to saving a phone number before texting someone. Peer information is stored in an `esp_now_peer_info_t` structure, which must be zero-initialized before use (`esp_now_peer_info_t peerInfo{};`) to avoid picking up random memory values:

```
uint8_t partnerMAC[6] = { 0x00, 0x70, 0x07, 0xA3, 0xDA, 0x14 };

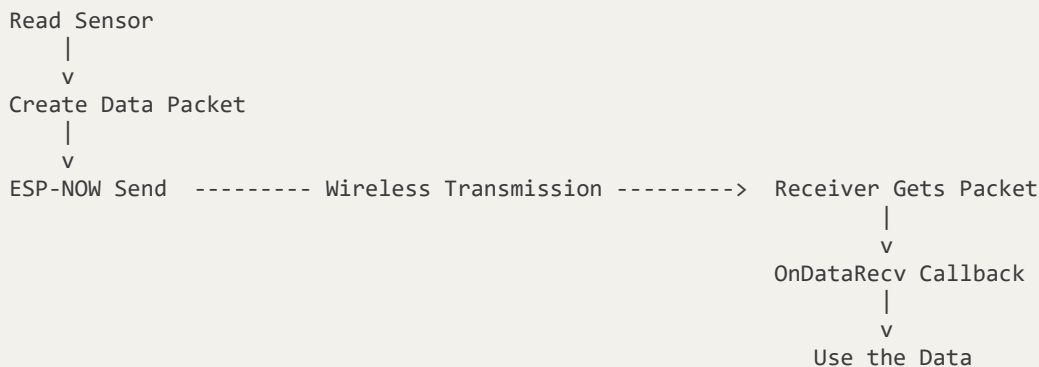
esp_now_peer_info_t peerInfo{};
memcpy(peerInfo.peer_addr, partnerMAC, 6);
peerInfo.channel = 0;
peerInfo.encrypt = false;
peerInfo.ifidx   = WIFI_IF_STA;

esp_now_add_peer(&peerInfo);
```

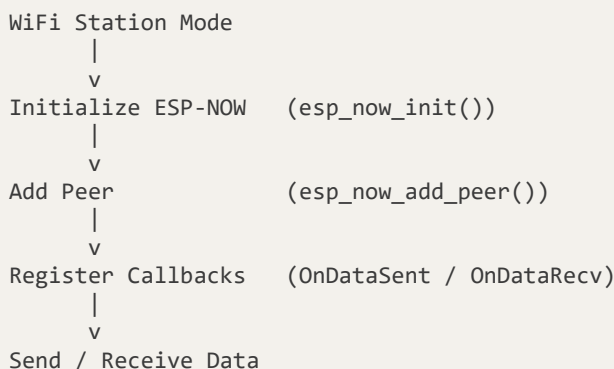
There is also a reserved broadcast address, `FF:FF:FF:FF:FF:FF`, which every nearby ESP-NOW receiver can pick up; real projects such as this one replace it with the partner's specific MAC address so only the intended board reacts.

3.4 How ESP-NOW Works — Communication Flow

End-to-end, a single packet's journey looks like this:



The required initialization order on both boards, every boot, is fixed:



Although ESP-NOW does not use a Wi-Fi router, it still relies on the ESP32's Wi-Fi radio hardware, so `WiFi.mode(WIFI_STA)` is called first to prepare that hardware in Station Mode — this does not connect the board to any network, it simply readies the radio.

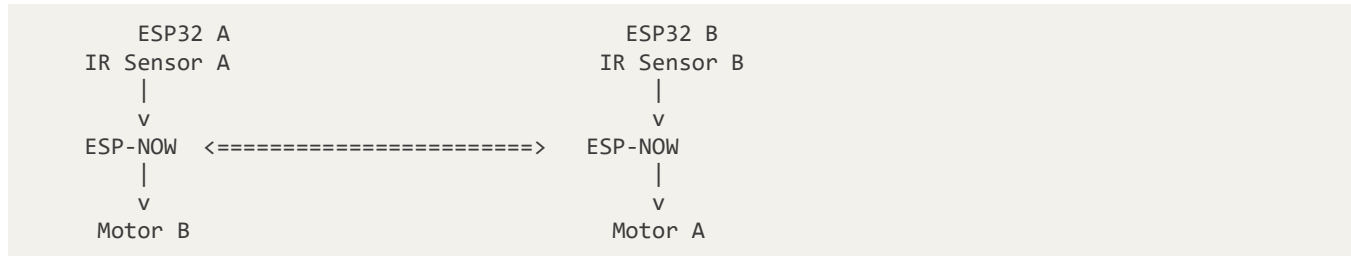
```
WiFi.mode(WIFI_STA);

if (esp_now_init() != ESP_OK)
{
    Serial.println("ESP-NOW initialization failed");
    return;
}
```

4. System Architecture

4.1 Peer-to-Peer (Transceiver) Design

Neither ESP32 in this project is a router, server, or master. Both boards run identical firmware logic and are simultaneously sender and receiver:



Board A reads IR Sensor A and sends the resulting command to Board B, which applies it to Motor A's own output... but from Board B's point of view, it is simultaneously reading IR Sensor B and sending its own command back to Board A for Motor B. The two directions run independently and concurrently in the same loop().

4.2 The Message Data Structure

ESP-NOW transmits raw bytes, not typed objects — so both boards must share an identical struct definition, compiled from the same Message.h header, or the byte layout on one side will not match the other:

```
// Message.h — shared by both boards
#pragma once
#include <Arduino.h>

typedef struct
{
    uint8_t senderID; // identifies which board sent this message (1 or 2)
    bool runMotor; // true = run partner's motor, false = stop it
    uint32_t timestamp; // millis() at time of send, used for the fail-safe check
} Message;
```

senderID distinguishes which board a message came from — e.g., ESP32 A sets msg.senderID = 1, ESP32 B sets msg.senderID = 2. runMotor is the literal command (true = run, false = stop). timestamp records when the packet was created via millis(), and is the value the fail-safe logic checks against.

4.3 Sending Data — esp_now_send()

```
Message msg;
msg.senderID = MY_ID;
msg.runMotor = !irSensor.obstacleDetected(); // true when path is clear
msg.timestamp = millis();

esp_now_send(partnerMAC, (uint8_t *)&msg, sizeof(msg));
```

Important Concept

esp_now_send() returning success does NOT mean the packet arrived — it only means the packet was successfully queued for transmission. Actual delivery confirmation is reported asynchronously through the OnDataSent() callback.

4.4 Receiving Data & Callback Functions

ESP-NOW is event-driven. Rather than polling "did I receive something?" in every loop iteration, the application registers two callbacks that ESP-NOW invokes automatically:

OnDataSent() — delivery status (optional, useful for debugging)

```
void OnDataSent(const uint8_t *mac, esp_now_send_status_t status)
{
    if (status == ESP_NOW_SEND_SUCCESS)
        Serial.println("Delivery Success");
    else
        Serial.println("Delivery Failed");
}

esp_now_register_send_cb(OnDataSent);
```

OnDataRecv() — essential, this is where the interlock logic reacts

```
volatile Message lastMsg;
volatile unsigned long lastReceivedTime = 0;

void OnDataRecv(const uint8_t *mac, const uint8_t *incomingData, int len)
{
    if (len != sizeof(Message)) return;    // reject malformed / wrong-sized packets

    memcpy((void *)&lastMsg, incomingData, sizeof(Message));
    lastReceivedTime = millis();           // "partner is alive" heartbeat marker
}

esp_now_register_rcv_cb(OnDataRecv);
```

Two rules govern both callbacks, and are enforced throughout this codebase:

- Never call `delay()` or perform blocking/heavy work inside a callback — keep it fast, non-blocking, and simple.
- The application never calls `OnDataSent()` / `OnDataRecv()` itself — ESP-NOW invokes them automatically whenever the corresponding hardware event occurs.
- Always validate `len` against `sizeof(Message)` before copying — receiving a wrongly-sized packet can corrupt memory.

5. Fail-Safe Design

5.1 Timeout Logic (2-Second Rule)

Wireless links can fail for several ordinary reasons — the partner board loses power, walks out of range, or its firmware crashes. A safety-critical interlock must never simply keep running the motor because "no STOP command arrived"; silence must be treated as a fault, not as permission to continue.

```
const unsigned long TIMEOUT = 2000;    // 2 seconds

void loop()
{
    // ... send our own state to the partner every iteration ...

    if (millis() - lastReceivedTime > TIMEOUT)
    {
        motor.stop();    // no packet in > 2s -> assume link/partner is down
    }
    else
    {
        motor.setState(lastMsg.runMotor);
    }
}
```

Embedded Design Principle

In safety systems, loss of communication should always resolve to the safest state. For this project:
Communication lost → Motor STOP.

5.2 Why millis() Instead of delay()

A tempting but incorrect shortcut is to `delay(2000)` and then stop the motor. During any `delay()` call the ESP32 cannot read sensors, cannot send its own heartbeat, and cannot service the ESP-NOW callback — every other task on the board is blocked for the full duration.

```
// WRONG — blocks the entire board for 2 full seconds
delay(2000);
motor.stop();
```

`millis()` returns a free-running counter without blocking anything, allowing continuous execution, non-blocking timing, and uninterrupted safety monitoring in parallel with every other task in `loop()`.

5.3 Heartbeat Messages

Sending a packet only when the IR sensor's state changes creates a blind spot: after one RUN message, the receiver has no way to tell whether its partner is alive and simply has nothing new to say, or has gone offline entirely.

```
IR detects object
|
Send STOP
|
... no further packets sent ...
|
Receiver cannot tell: is the partner alive or dead?
```

The fix is to transmit periodically regardless of whether the sensor state changed — e.g., every 100 ms — so the receiver's `lastReceivedTime` keeps advancing as long as the partner is alive:

```
RUN
RUN
```

```
RUN
RUN    <- steady stream every ~100 ms confirms "communication alive"

If power is removed:
No packets  --(2 seconds)-->  STOP MOTOR
```

6. ESPNow Class Design (OOP Wrapper)

To keep main.cpp focused on the interlock logic (read sensor -> decide -> send -> apply received state), the raw ESP-NOW C API was wrapped in a reusable ESPNow class, consistent with the modular OOP approach used for the L298N motor driver in the previous project:

```
class ESPNow
{
public:
    ESPNow();

    bool begin();
    bool addPeer(const
uint8_t *peerAddress);
    bool send(const
uint8_t *peerAddress,
const uint8_t *data,
size_t size);

    void onReceive(void
(*callback)(const
uint8_t *, const uint8_t
*, int));
    void onSend(void
(*callback)(const
uint8_t *,
esp_now_send_status_t));
};
```

Function	Responsibility
begin()	Set Wi-Fi station mode and initialize ESP-NOW
addPeer()	Register the partner board's MAC address as a trusted peer
send()	Transmit a raw byte buffer (the Message struct) to a peer
onReceive()	Register the application's receive callback
onSend()	Register the application's send/delivery-status callback

main.cpp then reduces to a small, readable safety loop: read the local IR sensor, build a Message, call espNow.send(), and separately check the fail-safe timeout before driving the L298N motor with the most recently received command.

7. Wiring & Hardware Setup

Each board in the pair reuses the Project 3 L298N motor driver wiring (12V supply, common ground with the ESP32, ENA on a PWM pin, IN1/IN2 on digital outputs) and adds an IR obstacle sensor wired to a digital input pin, plus onboard LEDs for RUN / STOP / fail-safe status.

ESP32 (Board)	L298N Driver	DC Motor
[PWM GPIO]	-----> ENA	
[GPIO]	-----> IN1	
[GPIO]	-----> IN2	
[GND]	-----> GND	<----- shared with 12V motor supply GND

OUT1/OUT2 -----> Motor terminals

ESP32 (Board)	IR Sensor
[Digital IN] <-----	OUT (obstacle detected = LOW/HIGH per module)
[3.3V/5V] ----->	VCC
[GND] ----->	GND

Pre-Power Checklist

Common ground verified between each ESP32 and its own L298N ($\approx 0 \Omega$). ENA jumper physically removed before attaching the PWM line. IR sensor GND shares the same reference as the ESP32 and motor driver. Both boards' MAC addresses double-checked against the values hardcoded in config.h before powering on.

A full labeled schematic and bench photographs for both boards are included as the required wiring diagram in the project's GitHub repository, alongside the PlatformIO source tree shown in Section 2.4.

8. Key Technical Concepts / Lessons Learned

ESP-NOW

- Enables direct ESP32-to-ESP32 communication without any router, access point, or internet connection.
- Addresses devices by MAC address rather than IP address.
- Both devices can send and receive simultaneously — there is no fixed client/server role.

Callbacks

- OnDataSent() and OnDataRecv() are called automatically by the ESP-NOW stack, never by application code directly.
- Callbacks must be fast and non-blocking — no delay(), no heavy processing.

Data Structures

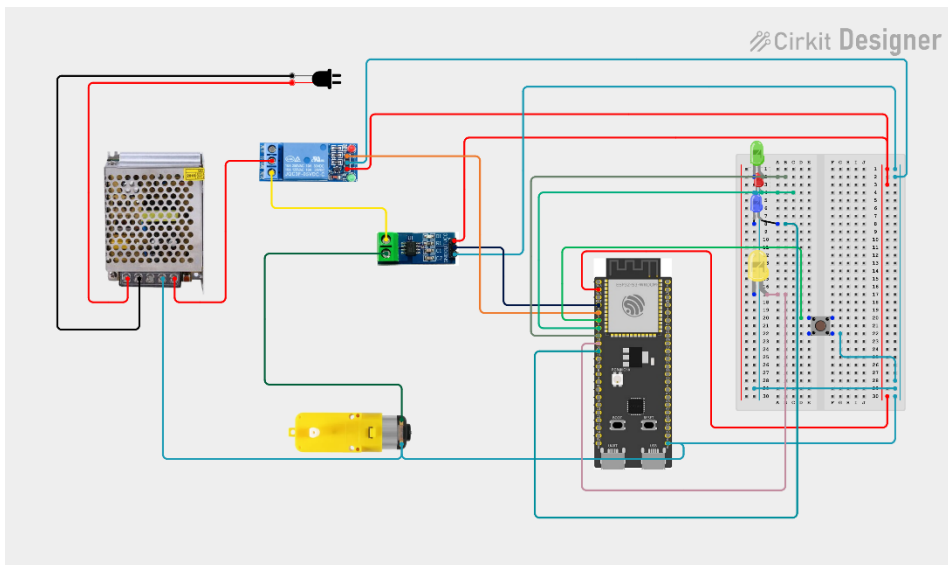
- Sender and receiver must use byte-for-byte identical structs, since ESP-NOW transmits raw bytes with no built-in serialization.

Fail-Safe

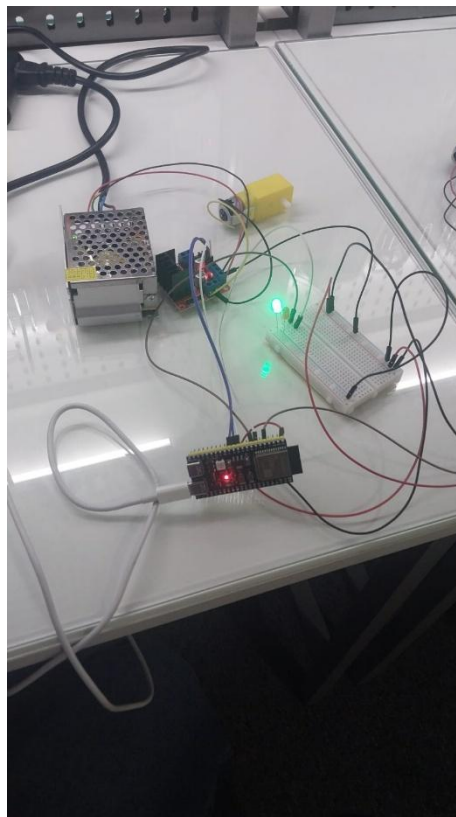
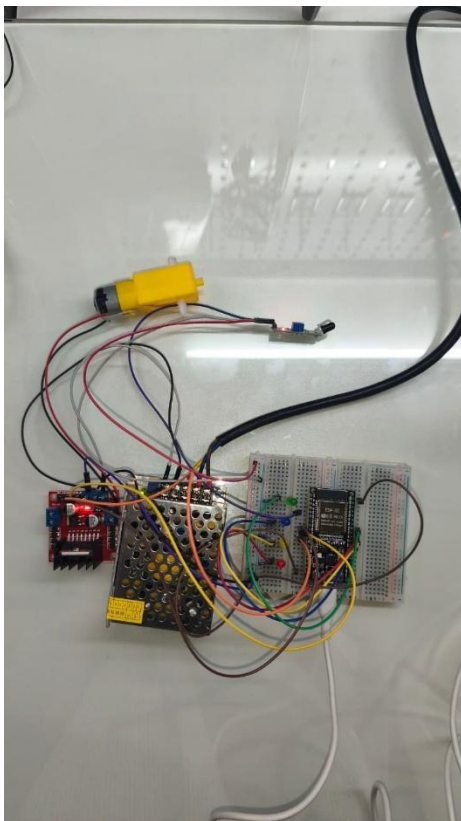
- Communication must be monitored continuously using millis(), never delay().
- Losing communication for more than 2 seconds must stop the motor — silence is treated as a fault condition, not a "do nothing" state.

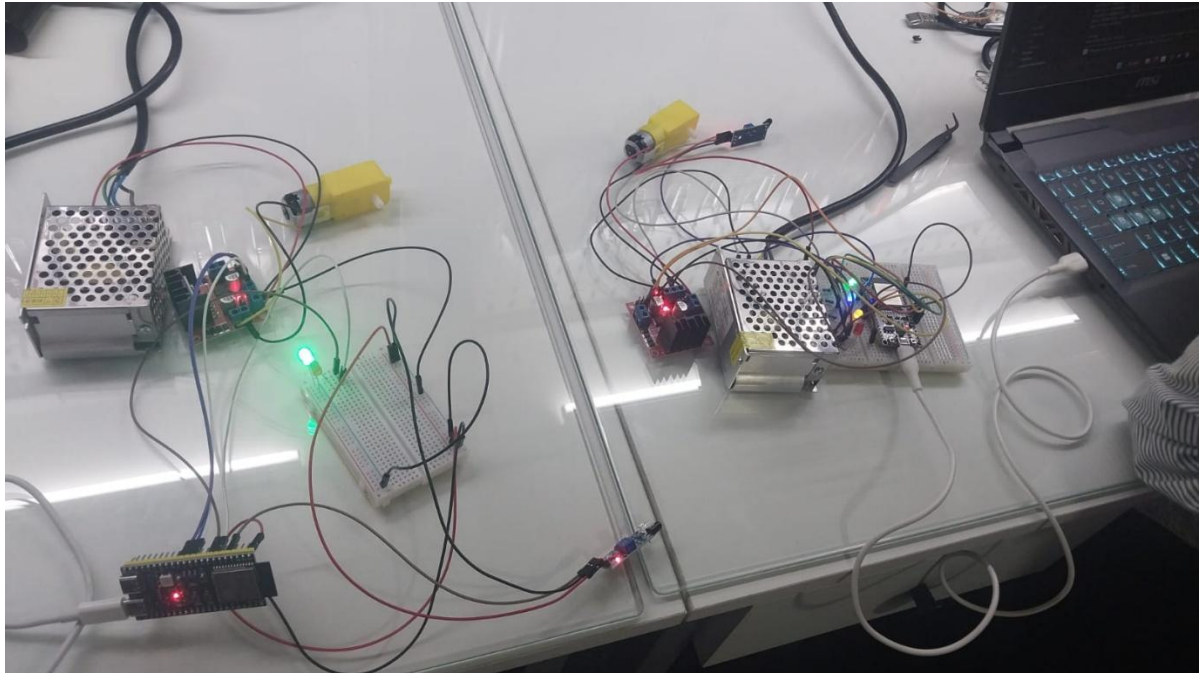
9. Schematic & Photos

Schematic



Photos





10. References

Sources consulted during this session, in order of use:

- [*Getting Started with ESP-NOW \(ESP32 with Arduino IDE\) | Random Nerd Tutorials*](#)
- [*ESP-NOW: The Ultimate Guide to Fast, Low-Power Communication with ESP32 & ESP8266*](#)
- [*cdn.bookee.app/files/pdf/book/en/developing-iot-projects-with-esp32.pdf*](#)
- [*Electronics Projects with the ESP8266 and ESP32 2020 Cameron.pdf*](#)
- [*ESP-NOW Two-Way Communication Between ESP32 Boards | Random Nerd Tutorials*](#)

End of Report